

**CIRCUITOS
DIGITAIS**



08/2025

Cartesian Game

CRIADO POR:

Marcos Pereira da Silva

INTRODUÇÃO

01

OBJETIVO

02

DESCRIÇÃO DO PROJETO

03

COMPONENTES PRINCIPAIS

04

TESTES

05

01 - INTRODUÇÃO

Este relatório documenta o desenvolvimento e a validação de um sistema digital no simulador Logisim ITA (versão 2.16.2.0.jar), denominado "Cartesian Game". O projeto permite posicionar um ponto em uma matriz de LEDs 15×15 a partir de um joystick (entradas X e Y de 4 bits), exibindo também as coordenadas em displays de 7 segmentos. Esse projeto é a continuação do projeto disponibilizado pelo professor no documento Projeto_Cartesian_Game_Logisim.

02 - Objetivo

O projeto tem como motivação consolidar os conhecimentos em lógica combinacional por meio do jogo proposto, exercitando conhecimentos como criação de tabela verdade, álgebra booleana, mapa de Karnaugh (ou K-map) e Logisim com seus componentes.

Para entrega é esperado os seguintes componentes:

1. Projetar demultiplexadores $4 \rightarrow 15$ e $15 \rightarrow 15$ para seleção de linhas e colunas de uma matriz de LEDs.
2. Projetar multiplexadores $4 \rightarrow 15$ e $15 \rightarrow 15$.
3. Implementar um somador de 4 bits.
4. Implementar um subtrator de 4 bits utilizando o complemento de 2.
5. Implementar um comparador que suporta entradas de 4 bits.
6. Desenvolver lógica de comparação para separar dígitos de dezena e unidade de valores binários superiores a 9 para o eixo X.
7. Construir conversores de binário para sinais de segmento de display, cobrindo os dígitos de 0 a 9.
8. Integrar todos os blocos em um circuito coeso, validando seu funcionamento por simulações em pontos de teste definidos.

03 - Descrição do Projeto

O projeto foi organizado em quatro blocos principais:

1. Demultiplexadores $4 \rightarrow 15$ e $15 \rightarrow 15$ (Linhas e Colunas): Cada demux recebe um vetor de 4 bits (entrada) e ativa uma das 16 saídas. A expressão de cada saída Y_i foi obtida a partir dos mintermos correspondentes à combinação desejada e implementada com portas AND.
2. Somador/Subtrator de 4 bits: Utilizando a estratégia para criação de um somador vista em aula, criei o subtrator fazendo a soma da entrada A com o complemento de 2 da entrada B. O complemento de dois foi obtido invertendo todos os bits de Y com portas NOT e somando o resultado com 1.
3. Bloco Separador Decimal: Uma vez obtido o valor binário de até 4 bits, um comparador combinacional analisa se o valor excede 9 (1001_2). Se maior, gera-se o bit de dezena = 1 e subtrai-se 10 (1010_2) do valor original para a unidade; caso contrário, dezena = 0 e unidade = valor original. A comparação foi feita por simplificação de expressões que cobrem os mintermos de 10 a 15.
4. Conversor para Display de Sete Segmentos: Para cada segmento, desenhou-se a tabela de números os quais o segmento será acionado. Cada segmento (a, b, c, d, e, f, g) teve sua função booleana derivada de mapas de Karnaugh, implementada com portas AND, OR e NOT.

Na fase de integração, foram utilizados dois demultiplexadores, um $4 \rightarrow 15$ e outro $15 \rightarrow 15$, um para a coordenada X e outro para a coordenada Y, ambos recebendo sinais diretamente do joystick para criar o componente GRAFICO. A saída desses demuxes foi conectada às linhas e colunas da matriz de LEDs 15×15 , possibilitando o acionamento preciso do LED correspondente à posição determinada pelas entradas. Simultaneamente, os valores binários das coordenadas fornecidas pelo joystick foram processados por blocos lógicos adicionais (SEPARADOR_X e SEPARADOR_Y) para realizar a separação dos dígitos de dezena e unidade, os quais foram então convertidos por circuitos lógicos (CONVERSOR_SEGMENTOS) para sinais adequados aos displays de sete segmentos. Dessa forma, o circuito final integra o controle visual da matriz e a exibição decimal das coordenadas de forma coesa e modular, favorecendo a clareza na visualização e manutenção do sistema.

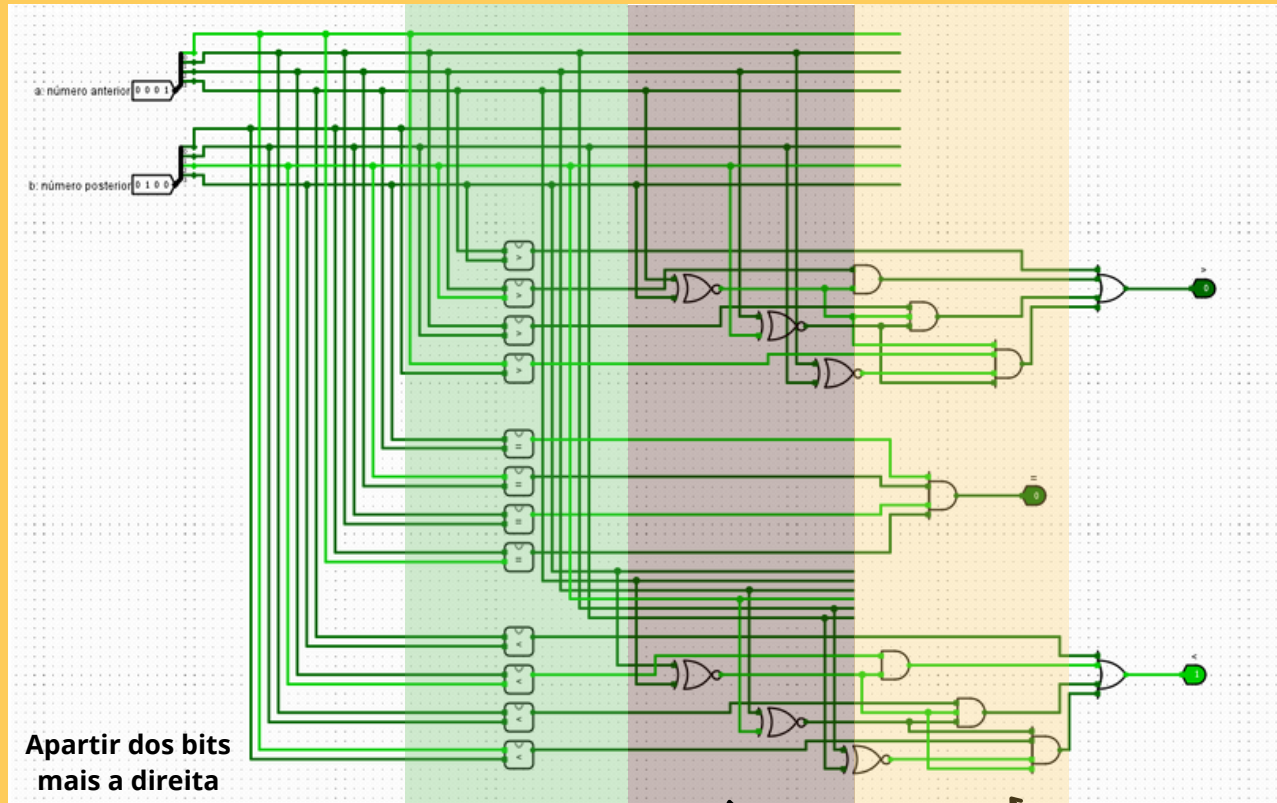
04 - ETAPAS DE IMPLEMENTAÇÃO

A construção do sistema seguiu uma abordagem semelhante ao desenvolvimento orientado a testes (TDD), onde cada componente foi especificado a partir de um conjunto de comportamentos esperados antes de ser efetivamente construído no Logisim. Inicialmente, foram elaborados testes conceituais para os subcircuitos, definindo para cada combinação de entrada qual seria a saída correta. Essas especificações permitiram orientar a modelagem das circuitos testando por feature em cada circuito.

1. Modelagem de Funções: Para cada subcircuito, elaborou-se a tabela-verdade com base no comportamento esperado e aplicou-se a técnica de mapas de Karnaugh para simplificação, obtendo-se expressões booleanas mínimas.
2. Testes Unitários: Implementei, para cada subcircuito, testes para validar no momento da criação de cada circuito se as saídas atendiam às previsões da tabela-verdade.
3. Desenvolvimento de Subcircuitos: criou-se individualmente cada bloco (demux, somador/subtrator, comparador, conversor), orientando-se pelas especificações previamente testadas. Após implementação, cada circuito foi confrontado com seus testes para validação imediata.
4. Integração dos Blocos: Após validação isolada, os subcircuitos foram conectados, substituindo os componentes já existentes no logisim.
5. Testes de Integração: Foram criados testes automatizados para validar se o projeto inteiro está atendendo os requisitos, simulando mudanças no joystick.

04 - COMPONENTES PRINCIPAIS

Comparador



1

Faz a verificação de cada operador lógico.

Para cada operador lógico, verifica se, para o bit X_i , os bits X_{i-1} das duas entradas são iguais.

2

Para cada operador lógico, verifica se, para o bit X_i , os bits $X_0, x_1, \dots, X_{i-1}$ das duas entradas são iguais e a comparação ($>$, $<$ ou $=$) é o caso.

3

04 - COMPONENTES PRINCIPAIS

Comparador

>

A	B	S_2
0	0	0
0	1	0
1	0	1
1	1	0

$$A \cdot \bar{B}$$

<

A	B	S_2
0	0	0
0	1	1
1	0	0
1	1	0

$$\bar{A} \cdot B$$

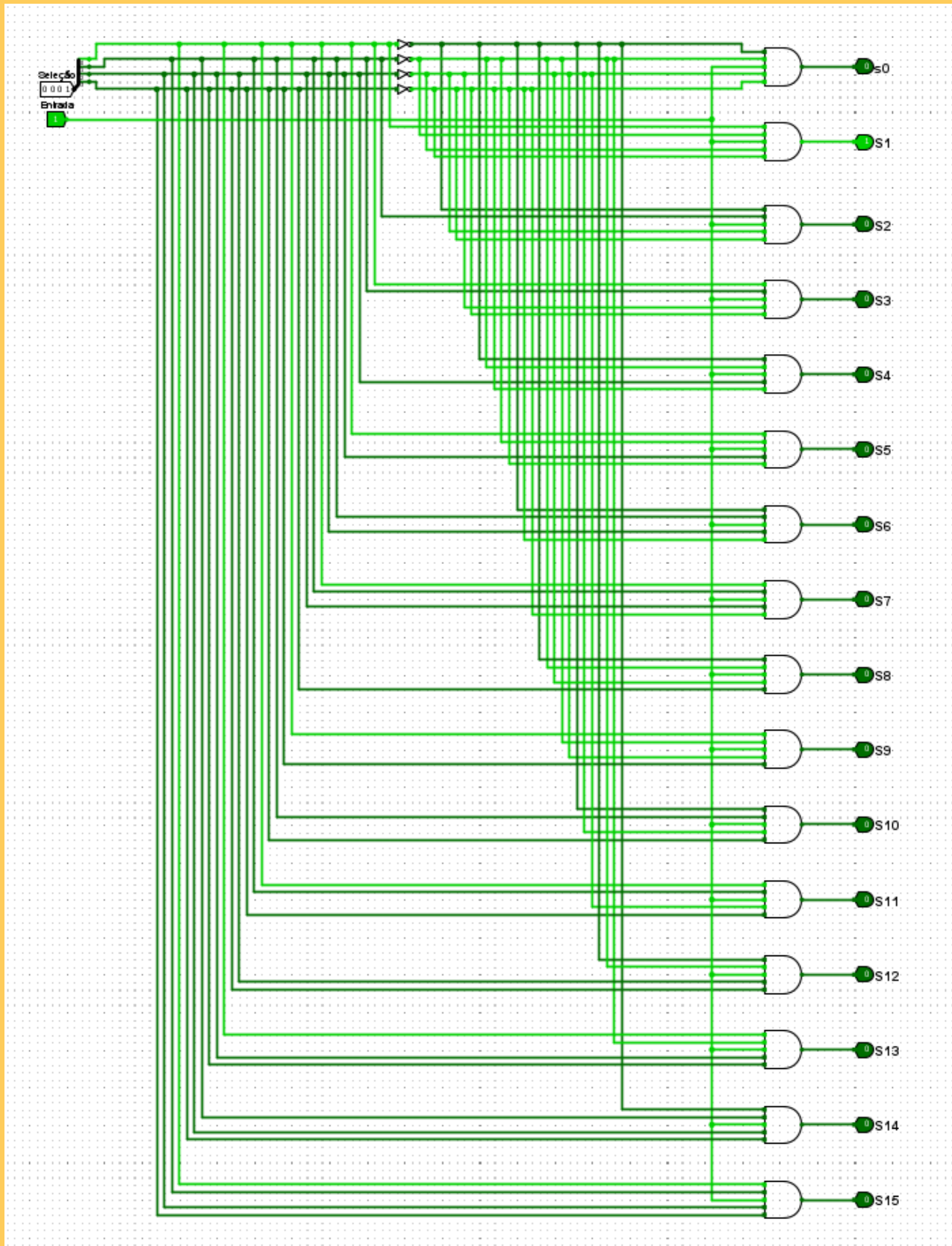
=

A	B	S_2
0	0	1
0	1	0
1	0	0
1	1	1

$$A \cdot B + \bar{A} \cdot \bar{B}$$

04 - COMPONENTES PRINCIPAIS

Demultiplexador



Mapeia cada entrada [0...15] em binário para cada uma das saídas.

04 - COMPONENTES PRINCIPAIS

Demultiplexador

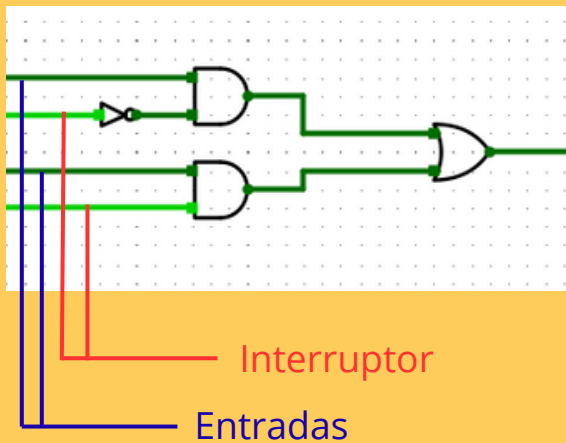
DMX

Hex	A	B	C	D	S _i
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
A	1	0	1	0	1
b	1	0	1	1	1
c	1	1	0	0	1
d	1	1	0	1	1
e	1	1	1	0	1
F	1	1	1	1	1

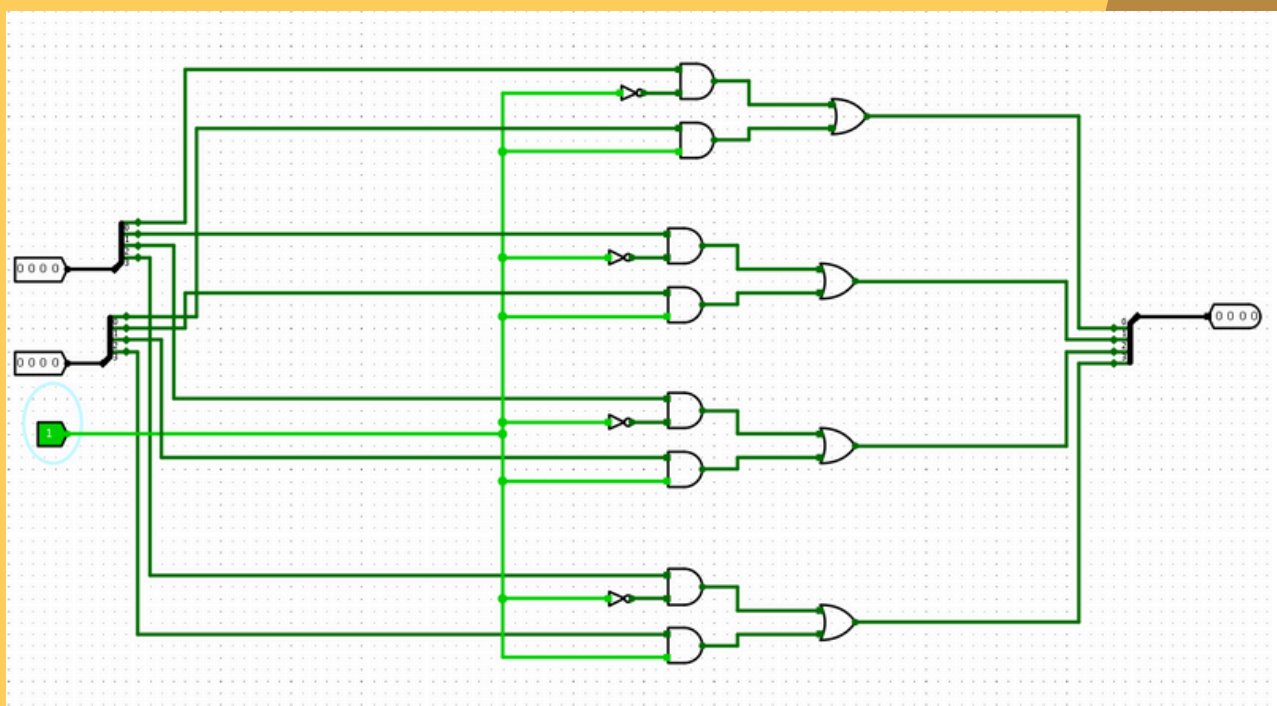
$\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}CD +$
 $\bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}BC\bar{D} + \bar{A}BCD +$
 $AB\bar{C}\bar{D} + AB\bar{C}D + ABC\bar{D} + ABCD$

04 - COMPONENTES PRINCIPAIS

Multiplexador



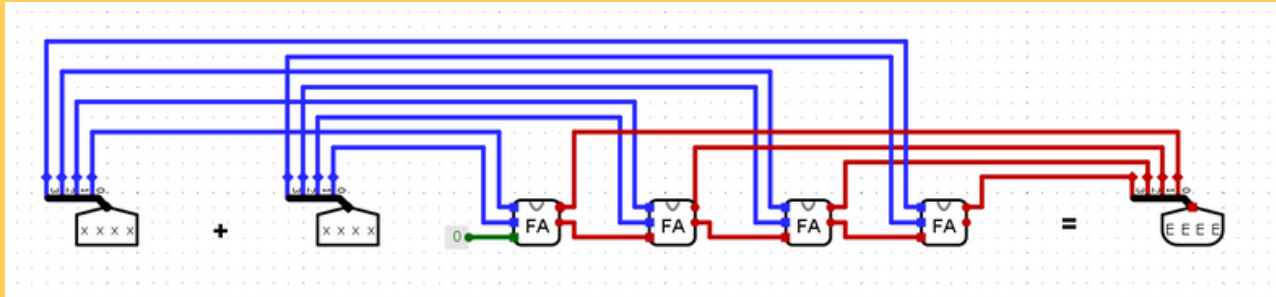
Dado duas entradas de 1 bit e um bit interruptor, redireciona somente um dos bits de entrada para a saída.



Repete a validação para 1 bit 4 vezes para a versão de 4 bits do Mux.

04 - COMPONENTES PRINCIPAIS

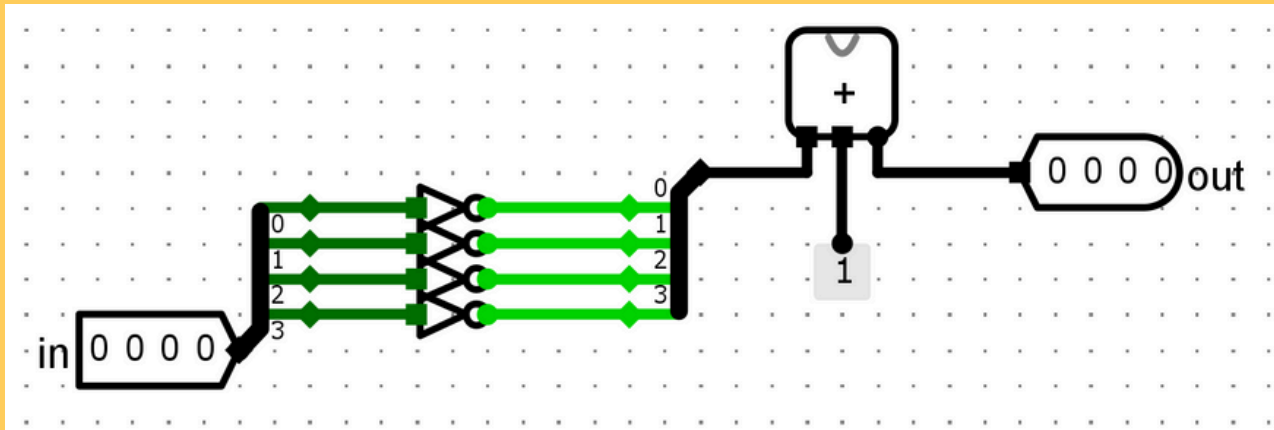
SOMADOR



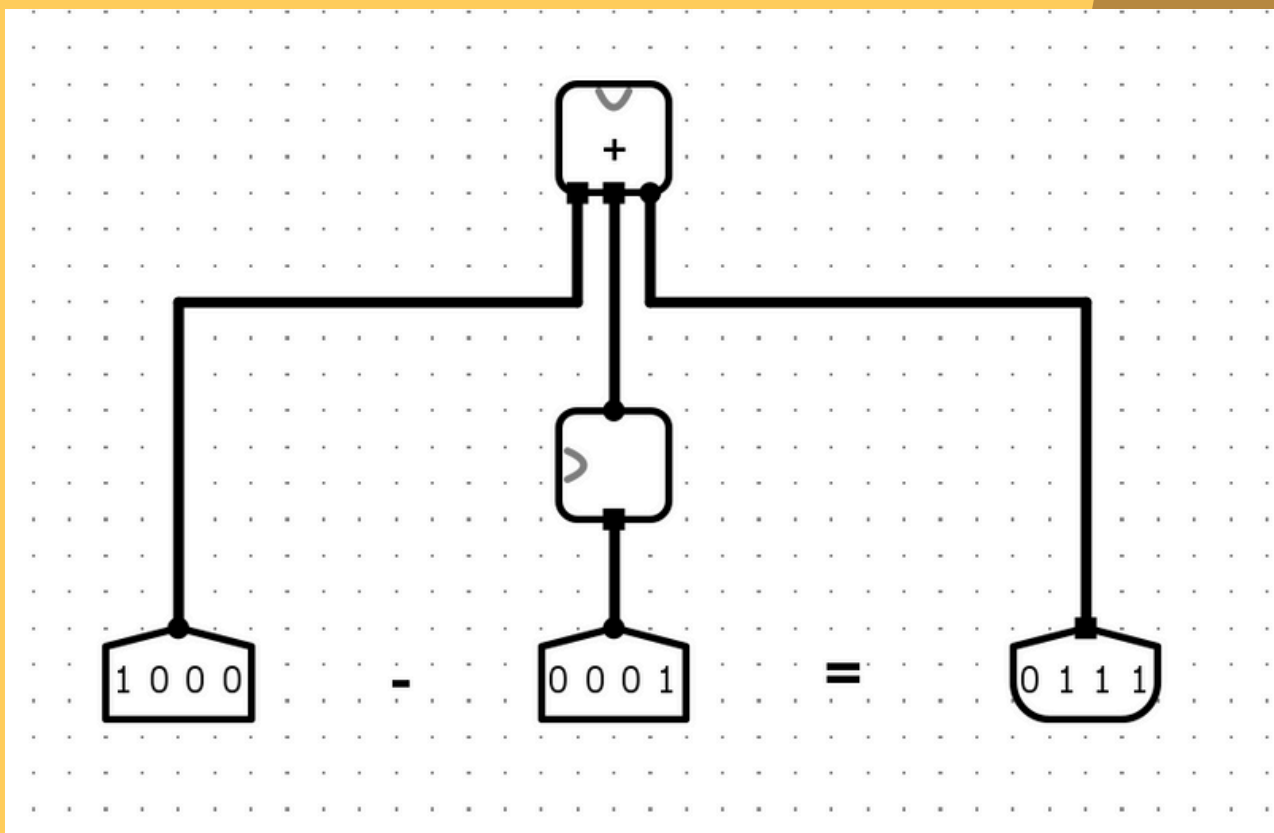
Concatenação de 4 full adders como visto em aula.

04 - COMPONENTES PRINCIPAIS

SUBTRATOR



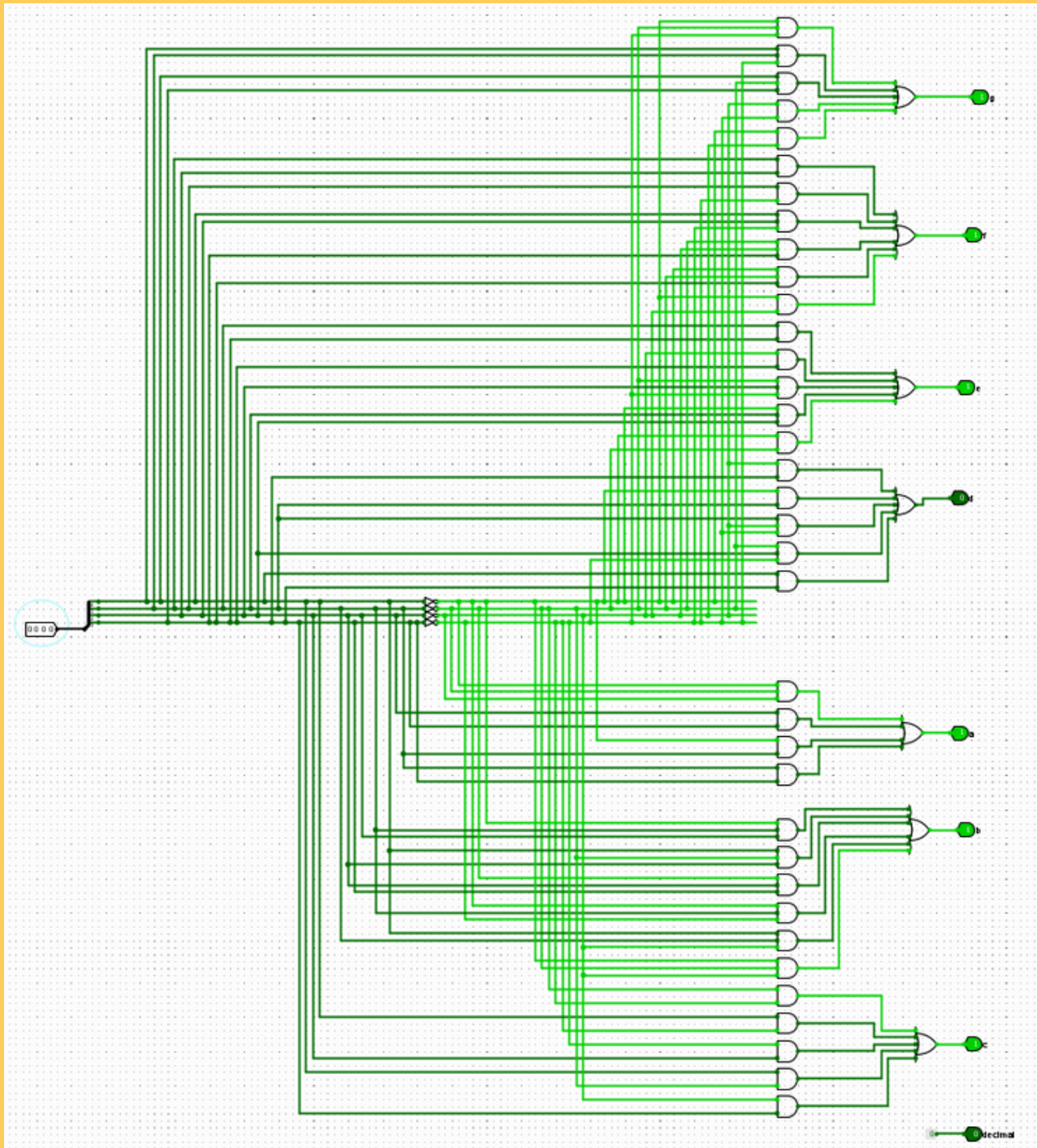
Complemento de 2 criado invertendo todos os bits da entrada e somando mais 1.



Subtração completa aplicando o complemento de 2 na segunda entrada.

04 - COMPONENTES PRINCIPAIS

CONVERSOR DE SEGMENTOS



04 - COMPONENTES PRINCIPAIS

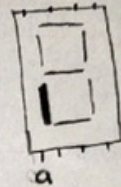
CONVERSOR DE SEGMENTOS

segmento **a**

Hex	A	B	C	D	a
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	0
A	1	0	1	0	1
b	1	0	1	1	1
C	1	1	0	0	1
d	1	1	0	1	1
E	1	1	1	0	1
F	1	1	1	1	1

AB/CD	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	1	1	1	1
10	1	0	1	1

$$\bar{B}\bar{C}\bar{D} + C\bar{D} + AB + AC$$



segmento **b**

Hex	A	B	C	D	b
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	0
A	1	0	1	0	0
b	1	0	1	1	1
C	1	1	0	0	1
d	1	1	0	1	1
E	1	1	1	0	1
F	1	1	1	1	0

AB/CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	1	1	0	1
10	1	0	1	0

$$\bar{B}\bar{C}\bar{D} + \bar{B}C\bar{D} + B\bar{C}\bar{D} + \bar{A}C\bar{D} + B\bar{C}\bar{D} + ABC$$



04 - COMPONENTES PRINCIPAIS

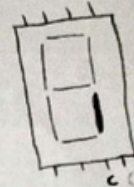
CONVERSOR DE SEGMENTOS

segmento [c]

Hex	A	B	C	D	c
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
A	1	0	1	0	1
b	1	0	1	1	1
C	1	1	0	0	0
d	1	1	0	1	1
E	1	1	1	0	0
F	1	1	1	1	0

	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	0	1	0	0
10	1	1	1	1

$$\bar{A}\bar{C} + \bar{A}D + \bar{A}B + \bar{C}D + A\bar{B}$$



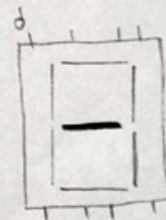
segmentos [d]

Hex	A	B	C	D	d
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	1
A	1	0	1	0	1
b	1	0	1	1	1
C	1	1	0	0	0
d	1	1	0	1	1
E	1	1	1	0	1
F	1	1	1	1	1

	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	0	1	1	1
10	1	1	1	1

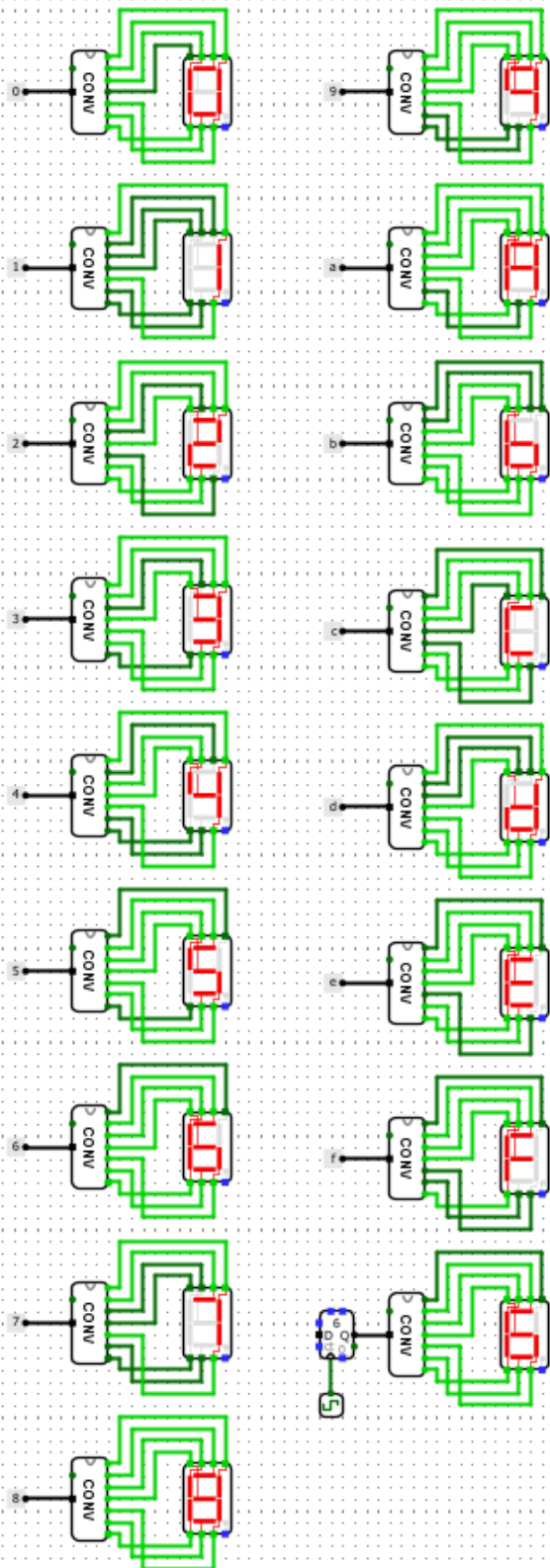
$$\bar{A}\bar{B} + A\bar{D}\bar{C} + A\bar{B} + \bar{A}\bar{B}C + C\bar{D}$$

$$\bar{A}\bar{B}\bar{C} + AD + A\bar{B} + \bar{A}\bar{B}C + C\bar{D}$$



05 - TESTES

CONVERSOR DE SEGMENTOS



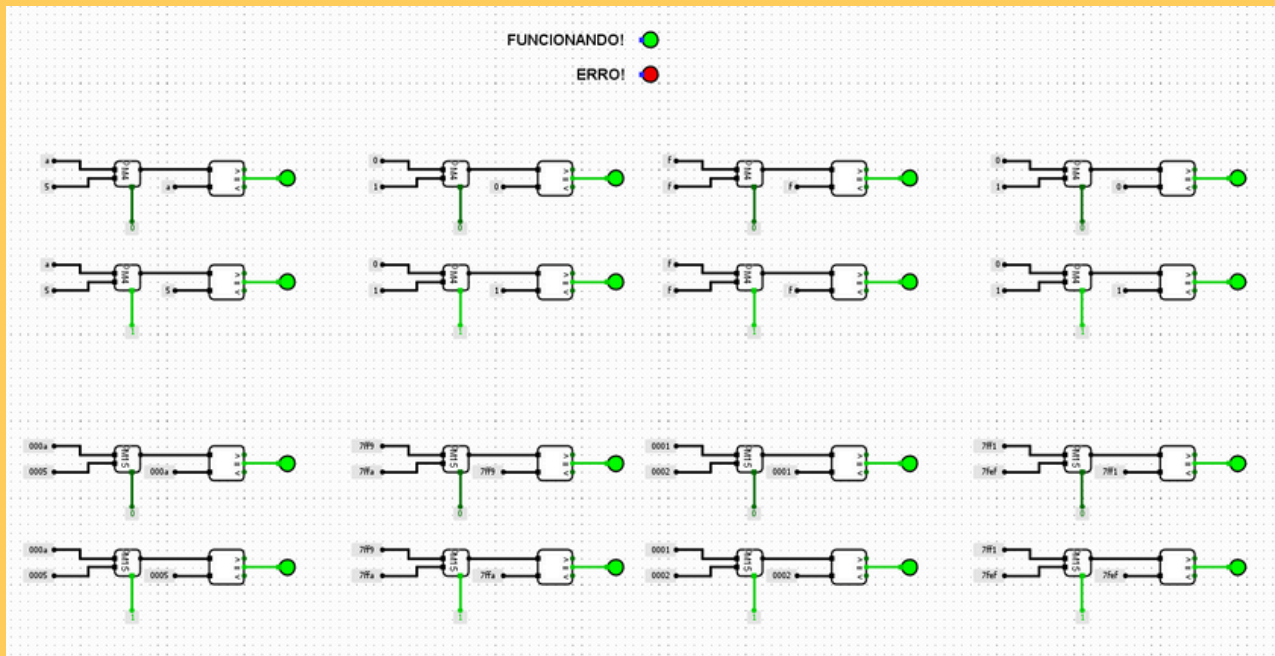
Esse teste verifica cada valor possível do conversor

CTRL + k para iniciar a simulação

Configurar a frequência de pulso para 4 Hz em: Simular > Frequência de pulso > 4 Hz

05 - TESTES

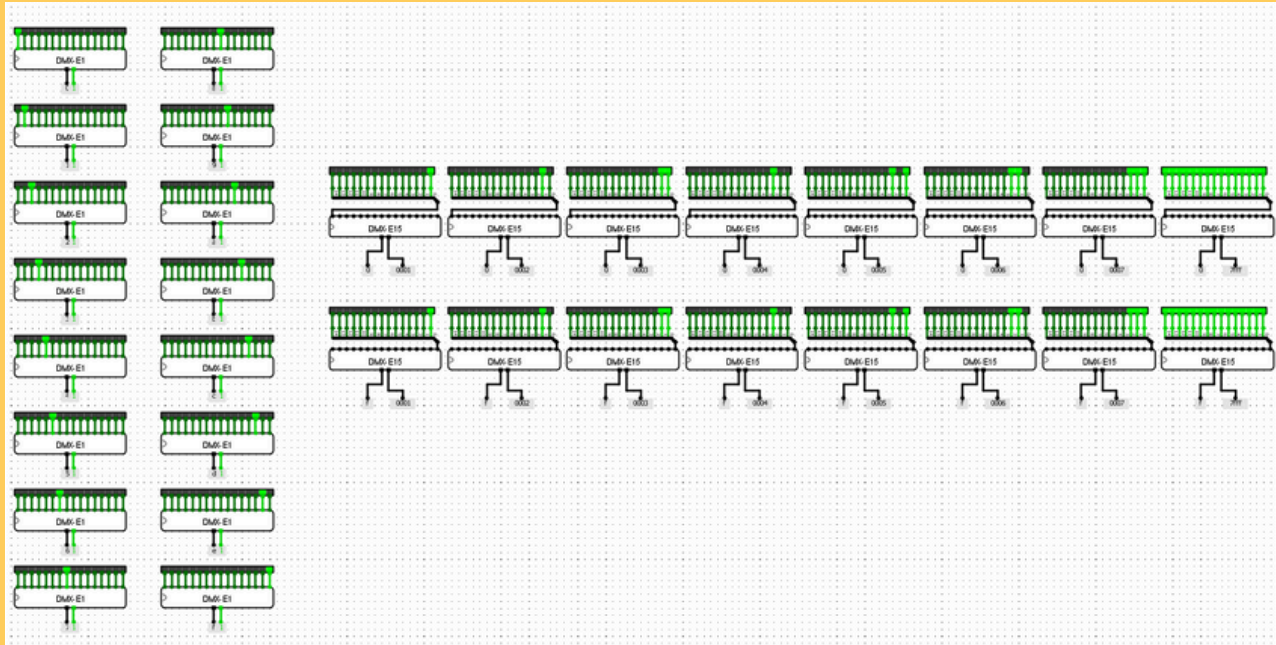
MULTIPLEXADOR



Alguns de teste para verificar as possibilidades do multiplexador

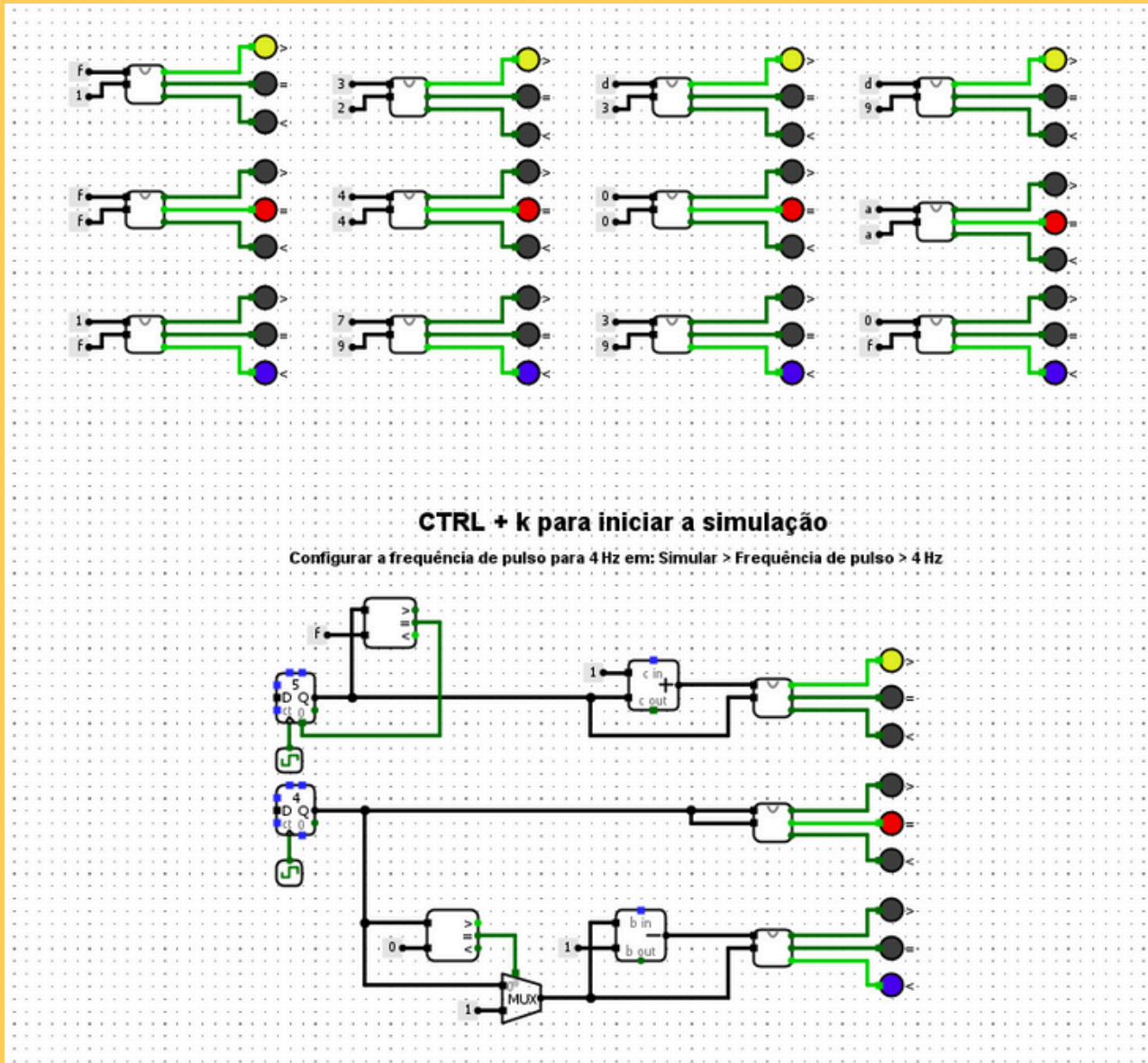
05 - TESTES

DEMULPLEXADOR



Alguns casos de teste para verificar as possibilidades do demultiplexador

05 - TESTES COMPARADOR



Casos de teste para verificar as possibilidades do comparador

- Os testes do '>' sempre deverão acender o led **amarelo**
- Os testes do '=' sempre deverão acender o led **vermelho**
- Os testes do '<' sempre deverão acender o led **azul**